

Multimodal Autonomous Agent Application

Multimodal Autonomous Agent Application

1. Concept Introduction
 - 1.1 What is an "Autonomous Agent"?
 - 1.2 Brief Description of Implementation Principles
2. Code Analysis
 - Key Code
 1. Agent Core Workflow (`largetmodel/utis/ai_agent.py`)
 2. Task Planning and LLM Interaction (`largetmodel/utis/ai_agent.py`)
 3. Parameter Processing and Data Flow Implementation (`largetmodel/utis/ai_agent.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configuring Online LLM
 - 3.2 Startup and Testing

1. Concept Introduction

1.1 What is an "Autonomous Agent"?

In the `largetmodel` project, the **Multimodal Autonomous Agent** is the most advanced form of intelligence. It no longer simply responds to a single user command, but is able to **autonomously think, plan, and continuously invoke multiple tools to complete tasks in order to achieve a complex goal**.

The core of this function is the `agent_call` tool or its underlying **Tool Chain Manager** (`ToolChainManager`). When a user makes a complex request that cannot be completed with a single tool call, the autonomous agent is activated.

1.2 Brief Description of Implementation Principles

The autonomous agent implementation in `largetmodel` follows the industry-mainstream **ReAct (Reason + Act)** paradigm. Its core idea is to mimic the human problem-solving process, cycling between "thinking" and "acting".

1. **Reason**: When the agent receives a complex goal, it first invokes a powerful language model (LLM) to "think". It asks itself: "What should my first step be to accomplish this goal? Which tool should I use?" The LLM's output is not the final answer, but an action plan.
2. **Act**: Based on the LLM's reasoning results, the agent executes the corresponding action -- invoking `ToolsManager` to run the specified tool (such as `visual_positioning`).
3. **Observe**: The agent obtains the result of the previous action ("observation"), for example `{"result": "Found the cup, position at [120, 300, 180, 360]"}`.
4. **Reason Again**: The agent submits this observation result, along with the original goal, to the LLM for a second round of "thinking". It asks itself: "I have found the cup's position. Next, to know its color, what should I do?" The LLM may generate a new action plan, such as `{"thought": "I need to analyze the image of the cup's area to determine the color", "action": "seewhat", "args": {"crop_area": [120, 300, 180, 360]}}`.

This **Reason -> Act -> Observe** cycle continues until the original goal is achieved, at which point the agent generates and outputs the final answer.

2. Code Analysis

Key Code

1. Agent Core Workflow (`largemodel/utils/ai_agent.py`)

The `_execute_agent_workflow` function is the Agent's main execution loop, defining the core "Plan -> Execute" process.

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...

    def _execute_agent_workflow(self, task_description: str) -> Dict[str, Any]:
        """
        Executes the agent workflow: Plan -> Execute.
        """
        try:
            # Step 1: Task Planning
            self.node.get_logger().info("AI Agent starting task planning phase")
            plan_result = self._plan_task(task_description)

            # ... (Return early if planning fails)

            self.task_steps = plan_result["steps"]

            # Step 2: Execute all steps in order
            execution_results = []
            tool_outputs = []

            for i, step in enumerate(self.task_steps):
                # 2.1. Before execution, process data references in parameters
                processed_parameters =
self._process_step_parameters(step.get("parameters", {}), tool_outputs)
                step["parameters"] = processed_parameters

                # 2.2. Execute single step
                step_result = self._execute_step(step, tool_outputs)
                execution_results.append(step_result)

                # 2.3. If step succeeds, save its output for subsequent steps to
reference
                if step_result.get("success") and
step_result.get("tool_output"):
                    tool_outputs.append(step_result["tool_output"])
                else:
                    # If any step fails, abort the entire task
                    return { "success": False, "message": f"Task terminated
because step '{step['description']}' failed." }

            # ... Summarize and return final results
            summary = self._summarize_execution(task_description,
execution_results)
```

```
        return { "success": True, "message": summary, "results":
execution_results }
```

```
# ... (Exception handling)
```

2. Task Planning and LLM Interaction (`largemodel/utils/ai_agent.py`)

The core of the `_plan_task` function is to construct a sophisticated Prompt that leverages the large model's own reasoning ability to generate structured execution plans.

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _plan_task(self, task_description: str) -> Dict[str, Any]:
        """
        Uses the large model for task planning and decomposition.
        """
        # Dynamically generate available tool list and descriptions
        tool_descriptions = []
        for name, adapter in
self.tools_manager.tool_chain_manager.tools.items():
            # ... (Get tool description from adapter.input_schema)
            tool_descriptions.append(f"- {name}({params}): {description}")
        available_tools_str = "\\n".join(tool_descriptions)

        # Construct highly structured planning Prompt
        planning_prompt = f"""
As a professional task planning Agent, please decompose the user task into a
series of specific, executable JSON steps.

**# Available Tools:**
{available_tools_str}

**# Core Rules:**
1. **Data Passing**: When subsequent steps need to use output from previous
steps, you must use the `{{{steps.N.outputs.KEY}}}` format for reference.
    - `N` is the step ID (starting from 1).
    - `KEY` is the specific field name in the previous step's output data.
2. **JSON Format**: Must return JSON object strictly.

**# User Task:**
{task_description}
"""

        # Invoke large model for planning
        messages_to_use = [{"role": "user", "content": planning_prompt}]
        # Note: This calls the general text inference interface
        result = self.node.model_client.infer_with_text("",
message=messages_to_use)

        # ... (Parse JSON response and return step list)
```

3. Parameter Processing and Data Flow Implementation (`largemodel/utils/ai_agent.py`)

The `_process_step_parameters` function is responsible for parsing placeholders and implementing data flow between steps.

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _process_step_parameters(self, parameters: Dict[str, Any],
previous_outputs: List[Any]) -> Dict[str, Any]:
        """
        Parses parameter dictionary, finds and replaces all {{...}} references.
        """
        processed_params = parameters.copy()
        # Regular expression for matching {{steps.N.outputs.KEY}} format
placeholders
        pattern = re.compile(r"\{\{steps\\.(\\d+)\\.outputs\\.([+?])\\}\\}\\}")

        for key, value in processed_params.items():
            if isinstance(value, str) and pattern.search(value):
                # Use re.sub and a replacement function to handle all found
placeholders
                # The replacement function looks up and returns values from the
previous_outputs list
                processed_params[key] = pattern.sub(replacer_function, value)

        return processed_params
```

Code Analysis

The AI Agent is the "central brain" of the system, transforming the high-level, sometimes ambiguous tasks proposed by users into a series of precise, ordered tool calls. Its implementation does not depend on any specific model platform, but is built on a general, extensible architecture.

1. **Dynamic Task Planning:** The Agent's core capability lies in the `_plan_task` function. It does not rely on hardcoded logic, but dynamically generates task plans through interaction with the large model.
 - **Self-awareness and Prompt Construction:** At the start of planning, the Agent first checks all currently available tools and their descriptions. Then, it packages this tool information, user tasks, and strict rules (such as data passing formats) into a highly structured `planning_prompt`.
 - **Model as Planner:** This Prompt is sent to a general text large model. The model reasons based on the provided context and returns a JSON-formatted action plan containing multiple steps. This design is highly extensible: when tools are added or modified in the system, the Agent's planning capability automatically updates without modifying the Agent's own code.
2. **Tool Chain and Data Flow:** Real-world tasks often require multiple tools to collaborate. For example, "take a photo and describe it" requires passing the output of the "take photo" tool (image path) as input to the "describe" tool. The AI Agent elegantly implements this through the `_process_step_parameters` function.
 - **Data Reference Placeholders:** During the planning phase, the large model embeds special placeholders in parameter values that need data passing, such as

- ```
{{steps.1.outputs.data}}.
```
- **Real-time Parameter Replacement:** In the main loop of `_execute_agent_workflow`, before executing each step, `_process_step_parameters` is called. It uses regular expressions to scan all parameters of the current step, and once a placeholder is found, it finds the corresponding data from the previous steps' output list and performs real-time replacement. This mechanism is key to automating complex tasks.
3. **Supervised Execution and Fault Tolerance:** `_execute_agent_workflow` constitutes the Agent's main execution loop. It strictly follows the planned step sequence, executing each action in order and ensuring data is correctly passed between steps.
- **Atomic Steps:** Each step is treated as an independent "atomic operation". If any step fails to execute, the entire task chain immediately aborts and reports an error. This ensures system stability and predictability, avoiding continued execution in an erroneous state.

In summary, the general implementation of the AI Agent demonstrates an advanced software architecture: it doesn't solve problems directly, but builds a framework that allows an external, general reasoning engine (large model) to solve problems. Through the two core mechanisms of "dynamic planning" and "data flow management", the Agent can orchestrate a series of independent tools into complex workflows capable of completing advanced tasks.

## 3. Practical Operations

### 3.1 Configuring Online LLM

1. **First obtain an API Key from any of the platforms in the previous tutorials**
2. **Then update the key in the configuration file, open the model interface configuration file `large_model_interface.yaml`:**

```
vim
~/yahboom_ws/install/largemodel/share/largemodel/config/large_model_interfac
e.yaml
```

3. **Enter your API Key:**

Find the corresponding section and paste the API Key you just copied. Here we use Tongyi Qianwen configuration as an example.

```
large_model_interface.yaml

Tongyi Qianwen
qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key
qianwen_model: "qwen-vl-max-latest" # You can choose the model as needed,
e.g. qwen-turbo, qwen-plus
```

4. **Open the main configuration file `yahboom.yaml`:**

```
vim ~/yahboom_ws/install/largemodel/share/largemodel/config/yahboom.yaml
```

5. **Select the online platform to use:**

Change the `llm_platform` parameter to the platform name you want to use.

```
yahboom.yaml

model_service:
 ros__parameters:
 # ...
 llm_platform: 'tongyi' #Optional platforms: 'ollama', 'tongyi',
 'spark', 'qianfan', 'openrouter'
```

## 3.2 Startup and Testing

### 1. Start the `largemodel` main program:

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
```

### 2. Send text commands:

Open another terminal again and run the following command:

```
ros2 run text_chat text_chat
```

Then start typing text: "Based on the current environment, generate an image of a similar scene".

### 3. Observe results:

In the first terminal running the main program, you will see log output showing the system receiving text commands, invoking the `aiagent` tool, then providing a prompt to the LLM. The LLM will analyze the detailed steps of tool invocation. For example, with this question, it will first call the `seewhat` tool to capture the scene, then provide the image to the LLM for analysis. The parsed text will then be given to the LLM as content for new image generation.